

Avaliação de predição de desvios no HeapSort em RISC-V

com o simulador gem5

Jean C. C. Gondorek Jonathan G. Correa Kassiara R. Bosing Lucas S. Schuch Matheus
H. R. da Costa

Organização de Computadores — UFFS

- No primeiro trabalho de Organização de Computadores usamos o simulador **Venus** para avaliar um HeapSort implementado pelo grupo.
- Nesta segunda etapa o mesmo algoritmo é analisado sob a ótica de **microarquitetura**, com o simulador de ciclo de clock **gem5**.

- Tópico: **branch prediction** (predição de desvios), comparando duas técnicas:
 - **LocalBP** — preditor local simples;
 - **BiModeBP** — preditor global (bi-mode).
- Métricas: total de desvios condicionais avaliados e número de predições incorretas, além de tempo e IPC.
- Objetivo: entender o desempenho de cada técnica em RISC-V e identificar qual é mais eficiente para o caso analisado.

- HeapSort implementado em assembly RISC-V no Trabalho 1.
- Usa um **min-heap** (árvore binária armazenada em vetor).
- Duas etapas: *construção* do heap e *extração* sucessiva dos elementos.
- Rotina central: `heapify`, que compara um nó com seus filhos e restaura a propriedade do heap.

Por que HeapSort é interessante para branch prediction?

- **Laços regulares** (`build_loop`, `extract_loop`): padrão de desvio previsível.
- **Comparações dependentes dos dados** dentro de `heapify`: o resultado depende dos valores do vetor.
- O mesmo endereço de instrução pode alternar entre *taken* e *not-taken*, dificultando a predição.

Desafios de conversão do HeapSort para assembly

- **Build do gem5:** compilação demorada ($>2h$) e inconsistente entre máquinas $\rightarrow$ padronização em ambiente **Docker**.
- **Syscalls do Venus:** pseudo-instruções didáticas substituídas por `printf` e `malloc` da `libc`.
- **Recompilação estática:** `riscv64-linux-gnu-gcc -static heapSort_original.s -o heapSort_original.riscv`.

- Quando encontra uma instrução condicional, o processador não sabe imediatamente se o salto será realizado ou não.
- Se esperar descobrir, o pipeline fica parado, reduzindo o desempenho.
- **O que é branch prediction?** É uma técnica usada em processadores para tentar adivinhar o resultado de uma instrução de desvio antes que ela seja executada.
- Em arquiteturas como a RISC-V, isso é importante porque o processador usa pipeline, evitando *hazards* de controle.

- O preditor é consultado e, para cada branch, a CPU guarda uma previsão, se utilizando de padrões anteriores para definir ela:
 - **Taken** (salta) → começa a buscar instruções do endereço `label`;
 - **Not Taken** (não salta) → continua buscando a próxima instrução sequencial.
- Quando a condição é finalmente calculada:
 - se a previsão estava correta, o pipeline continua normalmente;
 - se estava errada, as instruções carregadas incorretamente são descartadas (*pipeline flush*) e o processador busca as instruções corretas.

Estática: sempre segue uma regra fixa sem olhar o que aconteceu antes

- Sempre prevê “taken”.
- Sempre prevê “not taken”.
- Se o salto for para trás (loop), prever Taken; se for para frente, prever Not Taken.
- É barata mas não precisa.

Dinâmica: aprende com as execuções anteriores e usa esse histórico para prever o próximo resultado

- Pode usar tabelas de bits, contadores de 2 bits, histórico global etc.
- É o método utilizado em processadores modernos.
- Exige mais hardware, mas é muito mais precisa.

- É um preditor relativamente simples que usa o histórico local de cada branch.
- Para cada instrução de desvio, mantém um histórico próprio.
- Observa como aquele branch se comportou nas últimas execuções.
- Usa esse histórico para prever a próxima execução.
- Boa precisão para loops e padrões repetitivos, mas requer mais armazenamento para guardar históricos individuais e não aproveita correlações entre branches diferentes.

Exemplo: `beq x3, x2, label` → TTTNTTTN

O preditor aprende que esse branch geralmente é tomado e ajusta sua previsão de acordo.

- Preditor mais sofisticado, criado para reduzir interferências entre branches com comportamentos opostos.
- Possui tabelas especializadas para branches que tendem a ser *taken*, branches que tendem a ser *not taken*, e uma tabela de escolha que decide qual das duas consultar.
- Possui muito mais previsibilidade e menos interferência entre branches, mas o consumo de hardware é bem maior.

Exemplo: Branch A quase sempre *Taken*; Branch B quase sempre *Not Taken*. Em um preditor simples, ambos podem disputar as mesmas entradas da tabela e prejudicar a precisão. O BiMode separa esses comportamentos, reduzindo esse efeito, conhecido como *aliasing*.

- Simulações em ambiente Docker (`gem5.opt + inorder.py`), parametrizado (via IA) com `argparse`: CPU, preditor, clock, caches L1/L2, hierarquia, núcleos, memória.
- Valores fixados na comparação dos preditores: CPU TIMING (em-ordem), 3 GHz, L1 32 kB, L2 256 kB, DDR3 1 GB.
- Comparação executada com o vetor padrão ($n = 6$) e, para validar em escala maior, também com $n = 100.000$ elementos (gerado pela IA em assembly puro a partir de `heapSort_gem5.s`).

Comparação de preditores

- CPU **TIMING** (pipeline em-ordem)
- **Cenário A** — LocalBP (local simples)
- **Cenário B** — BiModeBP (bi-mode global)
- Demais parâmetros fixos no par

Escalas testadas

- $n = 6$ ($\{12, 11, 13, 5, 6, 7\}$)
- $n = 100.000$ (validação em escala)
- Clock 3 GHz; L1 32 kB; L2 256 kB

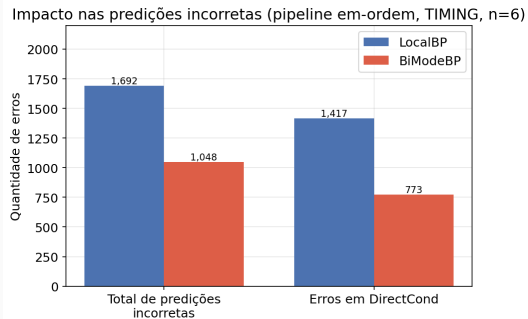
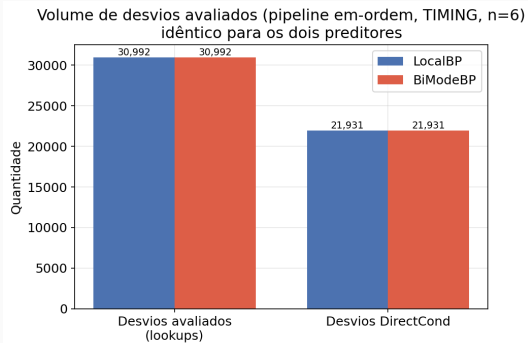
Observação: na CPU em-ordem (TIMING) a troca de preditor não altera o `simTicks` — esse modelo não penaliza ciclos por misprediction; por isso o ganho do BiModeBP aparece na contagem de erros, não no tempo simulado.

Extraídas de `m5out/stats.json` após cada simulação:

- `branchPred.BTBLookups` (consultas à Branch Target Buffer) — total de desvios avaliados;
- `branchPred.condIncorrect` (desvios condicionais incorretos) — previsões incorretas;
- `simTicks` — tempo total de simulação;
- **IPC** (instruções por ciclo) = `simInsts` / `core_numCycles` (instruções simuladas / ciclos do núcleo).

Taxa de erro global = `condIncorrect` / `BTBLookups` (previsões incorretas sobre o total de desvios avaliados).

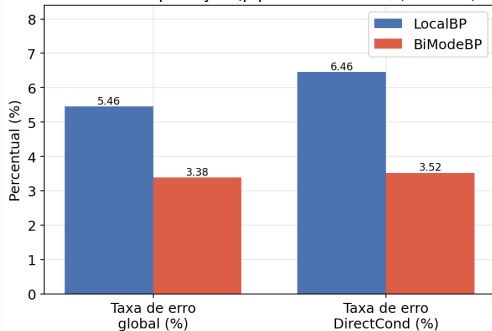
Resultados — volume e erros de predição ($n = 6$, em-ordem)



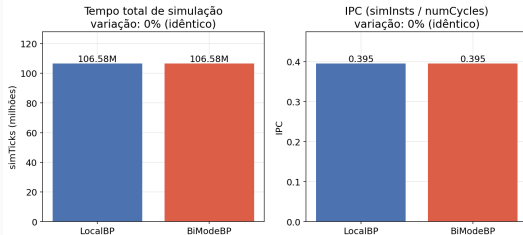
O total de desvios avaliados é idêntico (30.992) — o BiModeBP reduz as predições incorretas em $\sim 38\%$, concentrado nos desvios condicionais diretos (DirectCond) de heapify.

Resultados — taxas de erro e por que o tempo não muda

Taxas de erro de predição (pipeline em-ordem, TIMING, n=6)



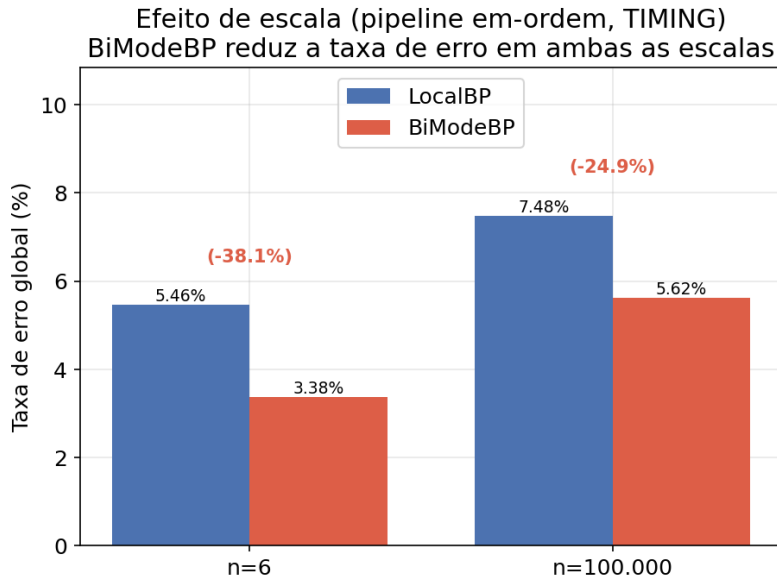
LocalBP vs. BiModeBP — pipeline em-ordem (TIMING, n=6)
simTicks e IPC não variam com o preditor neste modelo



O pipeline em-ordem mede *quantos* erros ocorrem, mas não aplica penalidade de ciclos por *misprediction*: simTicks e IPC ficam idênticos entre os preditores — o ganho do BiModeBP aqui é de **precisão**, não de tempo simulado.

Resultados — tabela comparativa ($n = 6$, CPU TIMING)

Métrica	LocalBP	BiModeBP	Var.
Desvios avaliados (lookups)	30.992	30.992	0%
Predições incorretas	1.692	1.048	−38,1%
Taxa de erro global	5,46%	3,38%	−2,08 p.p.
Erros em DirectCond	1.417	773	−45,4%
Taxa de erro DirectCond	6,46%	3,52%	−2,94 p.p.
simTicks	106,6 M (idêntico)		0%



Para o HeapSort em RISC-V no gem5, em pipeline **em-ordem (TIMING)**, o **BiModeBP** adapta-se melhor que o **LocalBP**:

- Redução de $\sim 38\%$ nas predições incorretas ($n = 6$), mantida em escala (24,9% em $n = 100.000$);
- Ganho concentrado nos desvios dependentes de dados de heapify, onde o mesmo PC alterna entre taken/not-taken a cada recursão;
- O `simTicks` é idêntico entre preditores — o modelo em-ordem não penaliza ciclos por *misprediction*: o ganho do BiModeBP é de **precisão**, não de tempo simulado.

- Os laços regulares (`build_loop`, `extract_loop`) são bem servidos por ambos os preditores; a diferença real aparece no `heapify`.
- **Recomendação:** o BiModeBP é preferível para cargas com mistura de laços regulares e comparações dependentes de dados, como o HeapSort.
- **Trabalho futuro:** repetir a comparação em pipeline fora de ordem para observar o efeito da predição sobre o tempo de execução.